

Autonomous LLM Agents for Active Directory Attack-Path Discovery: A Live-Collected BloodHound Benchmark

Evaluated on a real, live-collected Active Directory forest, not synthetic data.

Working draft. Results regenerate from `experiments/run_real.py` against an ingested BloodHound/SharpHound export.

Abstract

Active Directory (AD) attack paths, chains of individually-benign misconfigurations that escalate a low-privilege account to Domain Admin, are today surfaced by BloodHound's *rule-based* Cypher queries, which find exactly the shapes their queries encode. This paper asks whether a modern LLM agent can perform the human analyst's role instead: exploring the AD graph step by step, through a small set of query tools, and proposing a valid attack path on its own, with a mandatory self-verification step before it may answer. Unlike prior work on this codebase, which relied on a synthetic generator, every result here is measured against a **real, live-collected AD forest**: a two-domain Windows environment (GOAD-Light, deployed on disposable cloud VMs and destroyed after collection) enumerated with SharpHound, BloodHound-CE, and Certipy exactly as an operator would collect it in the field. Four models across three vendors (Google, OpenAI, Anthropic) are evaluated, spanning a roughly 100× price range, with real metered API cost recorded per run. The central finding is that cost and correctness are inversely related on this data: the free model achieves the highest correctness of the four (66.7%); the two most expensive models tested tie for the lowest (16.7% each, at up to 100×+ the cost of the mid-priced model that beats them); correctness across all four models is substantially lower than prior synthetic-graph results on the same architecture (16.7 to 66.7% versus 76 to 100%); and hallucination holds at 0% across all 24 runs and all three vendors. Every rate is reported with a 95% Wilson interval given how small the per-model samples are, and this paper discusses explicitly where its design (a rule-based-query baseline with no equivalent verification pass, a self-referential verifier, a fixed step budget on a graph an order of magnitude denser than earlier synthetic graphs) should and should not be trusted.

1. Introduction

BloodHound maps an AD forest into a graph of principals (users, groups, computers) and control edges (MemberOf, AdminTo, GenericAll, ForceChangePassword, and others), then runs pre-written queries such as “shortest path to Domain Admins.” Because it is rule-based, it finds exactly the patterns its queries encode; unusual-shaped chains still require a human to reason over the graph. This work measures whether an LLM agent can do that reasoning on **genuine collected data**, not a synthetic stand-in, and how it compares to the rule-based baseline and to itself across models of very different price.

A benchmark on synthetic, generator-planted graphs can show an agent is *capable* of attack-path reasoning under controlled conditions. It cannot show how that capability holds up against the messier structure of a real, ACL-dense AD forest, where GenericAll, WriteDacl, WriteOwner, and DCSync edges number in the hundreds and the shortest real path is rarely the only plausible-looking one. This paper reports what happens when the same agent architecture is pointed at data collected from an actual Windows Active Directory lab.

2. Background

2.1 Active Directory Attack Paths

An adversary who holds one low-privilege foothold typically escalates by discovering that the compromised principal can, directly or through group membership, control some other object, which in turn controls another, until a member of Domain Admins is reached. Each individual permission may be legitimate; the composition is what is dangerous.

2.2 BloodHound and Rule-Based Path Discovery

BloodHound collects the AD graph with SharpHound (or the BloodHound-CE collector) and stores it in Neo4j, then evaluates canned Cypher queries over control edges, canonically, shortest path to Domain Admins. Its recall is bounded by the edge types collected and the query patterns written; it is silent on anything outside that. This canonical shortest-path query, over canonical edges only, is used as the rule-based baseline throughout this paper.

2.3 LLM Agents and the ReAct Loop

An LLM agent interleaves natural-language reasoning with tool calls, observing each result before deciding the next action: the reason, act, observe pattern known as ReAct. Given only a foothold and a goal, the agent must query a node's neighbours, read its properties, form a hypothesis about the next hop, and verify. This closely resembles manual BloodHound analysis, which is why it is adopted as the architecture under test.

3. Live Data Collection

3.1 The Lab

GOAD-Light (Orange Cyberdefense's open-source Active Directory range, commit 992307a) was built as a two-domain forest, `sevenkingdoms.local` (parent, with ADCS) and `north.sevenkingdoms.local` (child, with a member server running IIS and MSSQL), on five disposable Vultr cloud VMs (a domain controller per domain, one member server, one workstation-capable box, and one Linux control node running Ansible). The build ran GOAD's own Ansible playbooks staged against pre-existing cloud VMs rather than Vagrant-provisioned ones, which surfaced several real integration issues undocumented for this deployment path: DNS self-registration on a freshly promoted child DC, a stale local Administrator credential surviving domain promotion, and API-gateway compatibility quirks across three different LLM providers (Section 5). None of these are benchmark artefacts; they are the ordinary friction of running this tooling against real infrastructure, and they are mentioned here because a paper claiming “real data” should be honest about how it was obtained.

The entire lab was destroyed (after a disk snapshot) at the end of collection. No part of it persists as running infrastructure, and no live or third-party network was touched at any point.

3.2 Collection

SharpHound, BloodHound-CE, and Certipy were run from a compromised-account vantage point, producing the standard collector JSON (users, groups, computers, domains, GPOs, containers, OUs, plus ADCS templates). The two domains' collections were merged into one graph and ingested through the same schema-normalising pipeline this codebase already used for synthetic data, so the agent, tools, and scorer are unchanged between synthetic and real runs; only the data source differs. The resulting graph has **147 nodes** (3 computers, 2 domains, 110 groups, 33 users) and **1,259 canonical edges**, dominated by ACL-style control edges (GenericAll 325, WriteDacl/WriteOwner 206 each, GenericWrite 202, AllExtendedRights 57, DCSync 4): roughly 6× denser per node than the synthetic graphs used in earlier development of this codebase, and structured very differently, almost entirely canonical ACL abuse rather than the inference-only tradecraft (Kerberoasting, unconstrained delegation, ADCS ESC1, credential exposure) that a synthetic generator can plant deliberately and evenly.

4. Agent, Verifier, and Scoring

The agent is a minimal ReAct loop over six tools, `search_node`, `query_outbound_edges`, `query_inbound_edges`, `get_node_properties`, `check_path_exists`, and `verify_path`, which it must call to self-check its proposed path before finishing; a rejected hop sends it back to repair the path rather than ship an invalid one. It is never shown the whole graph.

Verification means one thing everywhere in this codebase. Each consecutive pair of nodes on a proposed path must be a real edge in the collected graph (or, where applicable, a property-justified inference step), and the final node must be a Domain Admins group. A hop that is neither counts as a **hallucination**. This verifier is explicitly **internal to the system under test**: it checks a proposed path against the same graph the agent queried, using the same code path for both the agent's own self-check and the scorer's final judgement. This is a meaningful check for internal consistency, catching a model asserting an edge that is not in the collected data, but it is not an independent, external validation of the verifier's own logic, and it cannot detect an error common to both (Section 8).

5. Experimental Setup

Models. Four models across three vendors were chosen to span a roughly 100× price range on identical hardware access to the same graph, listed in Table 1.

Table 1. Models evaluated, by vendor, access route, and list price.

Model	Vendor	Route	List price (per M tok, in/out)
<i>gemini-flash-lite-latest</i>	Google	native, free tier	\$0 (rate-limited)
<i>gpt-4o-mini</i>	OpenAI	direct billing	\$0.15 / \$0.60
<i>claude-haiku-4-5</i>	Anthropic	direct billing	\$1.00 / \$5.00
<i>gpt-4o</i>	OpenAI	direct billing	\$2.50 / \$10.00

Start users (6, deliberately chosen, not randomly sampled). Four real path cases, `lord.varys` (a one-hop GenericAll to Domain Admins privilege-escalation), `catelyn.stark` and `robb.stark` (multi-hop routes into the child domain's Domain Admins), and two no-path controls, `jaime.lannister` (whose ACL chain reaches a domain-controller computer object, not the Domain Admins group) and `hodor` (a low-privilege account with no path at all), were used to test whether a model asserts a path where none exists. This deliberate selection, rather than random sampling, is itself a limitation discussed in Section 8.

Step budget. 35 reasoning steps, raised from an initial 15 after the denser real graph caused premature “ran out of steps” failures at the lower budget on cases every model could solve at 30+ steps. This adjustment is reported rather than silently tuned away.

Cost. OpenAI and Anthropic costs are real metered API billing (verified against each provider's own token-price table where the gateway does not echo cost directly); Gemini's free tier is \$0 with a request-rate ceiling. Total spend across all reported runs was **\$3.96**.

6. Results

Table 2. Overall results, 24 scored runs of 25 attempted (one attempt was dropped to a transient rate-limit).

Metric	Value
Runs scored	24 (of 25 attempted)
Correctness	37.5% [21.2 to 57.3]
Valid-path rate	8.3% [2.3 to 25.8]

Hallucination rate	0.0% [0.0 to 13.8]
Optimal of paths found	100.0% (2 of 2 found)
Beats rule-based baseline	0 of 8 advanced-required
Avg tool calls (solved)	14.9
Avg runtime (s)	117.9
Cost (USD)	\$0.165/run avg, \$3.96 total

Table 3. Results by model.

Model	Runs	Correctness (95% CI)	Halluc.	Avg tools	Avg time (s)	Avg cost
<i>gemini-flash-lite-latest</i>	6	66.7% [30.0 to 90.3]	0.0%	25.2	124.8	\$0.000
<i>gpt-4o-mini</i>	6	50.0% [18.8 to 81.2]	0.0%	6.5	14.3	\$0.003
<i>claude-haiku-4-5</i>	6	16.7% [3.0 to 56.4]	0.0%	19.7	88.1	\$0.305
<i>gpt-4o</i>	6	16.7% [3.0 to 56.4]	0.0%	16.7	244.4	\$0.352

Table 4. Failure-mode breakdown, 24 runs against one graph.

Correct	Hallucinated	Gave up (path existed)	Ran out of steps	Wrong path
9	0	7	6	2

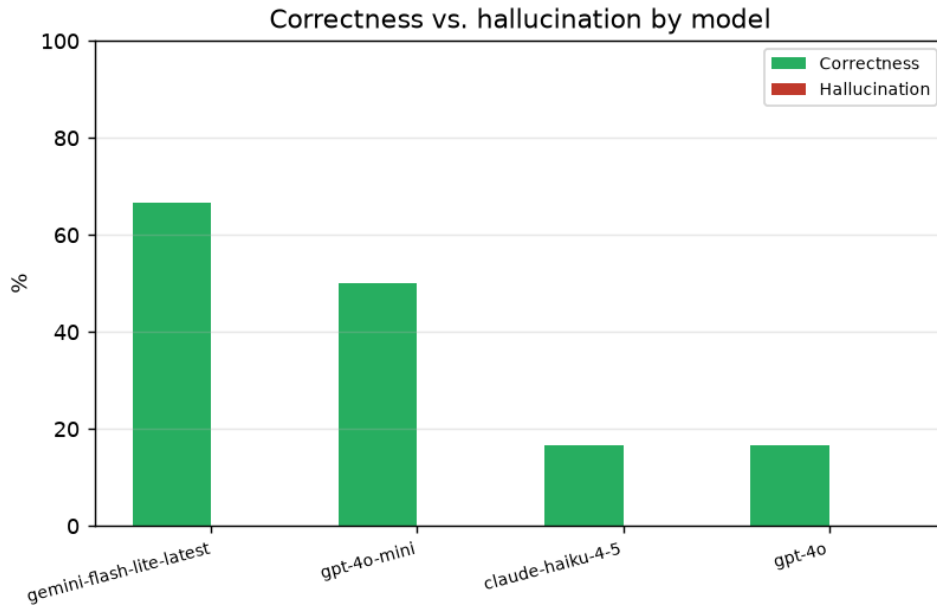


Figure 1. Correctness and hallucination rate by model.

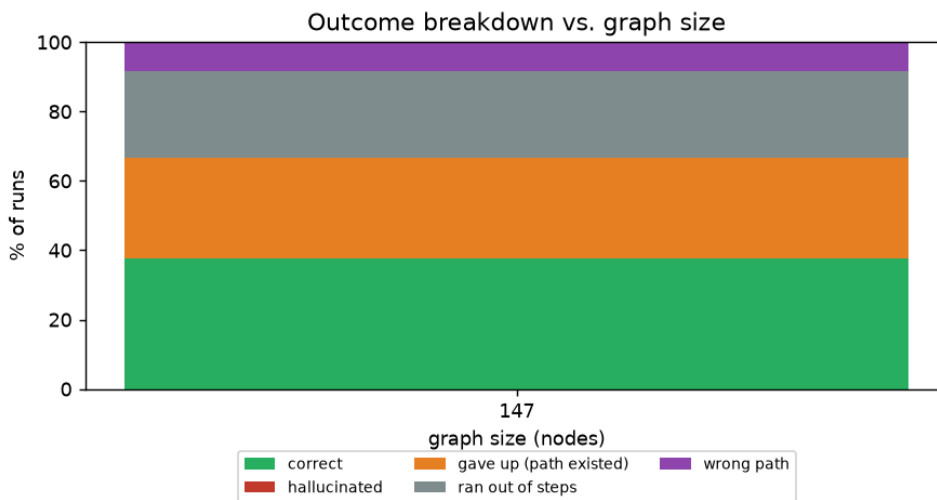


Figure 2. Outcome breakdown across all runs.

6.1 Reading it

Cost and correctness are inversely related on this data. The free model (gemini-flash-lite-latest) scored highest of the four (66.7%, Table 3); the two most expensive models tested, gpt-4o (\$2.50/\$10.00 per M) and claude-haiku-4-5 (\$1.00/\$5.00 per M), tied for lowest (16.7% each), both scoring well below the far cheaper gpt-4o-mini (\$0.15/\$0.60 per M, 50.0%). gpt-4o, specifically, took **17× longer per run on average** (244.4s versus 14.3s for gpt-4o-mini) while scoring worse, repeatedly re-querying the same region of the graph before giving up. This is read as a genuine signal on this data and this task, not a general claim about these vendors' models across all tasks: the sample per model is 6 runs, and the correctness intervals for the three non-Gemini models overlap substantially (Section 8).

Hallucination held at 0% across every model and every run. All 24 runs, across 3 vendors and a 100× price spread, produced zero hallucinated hops (Table 2); no model in this sample asserted a path the collected graph does not support. This is the result reported with the most confidence: even where a model failed to find a real

path (the dominant outcome at every price point, Table 4), it failed by giving up or running out of budget, not by inventing one.

Correctness is substantially lower than prior synthetic-graph work on this same agent/verifier architecture. Earlier synthetic benchmarks reported 76 to 100% correctness on generator-planted graphs of comparable node count. Here, on a real, ACL-dense collected forest, correctness ranges 16.7 to 66.7% across models: roughly half or less. The overall hallucination rate (0.0% [0.0 to 13.8]) and the prior synthetic rate (0% [0.0 to 9.0], from an earlier, differently-scoped sweep) are consistent with each other; correctness is where the real-versus-synthetic gap actually shows up.

The rule-based baseline was never beaten. 0 of 8 cases the true-reachability search flagged as requiring an inferred (non-canonical) step were solved by any model, versus 10 of 12 on prior synthetic graphs. The real GOAD-Light collection is overwhelmingly canonical-ACL-shaped rather than inference-shaped (Section 3.2); this is consistent with, and does not contradict, the synthetic finding: it says the opportunity to demonstrate this specific capability was much rarer on this real forest, not that the capability regressed. There were too few inference-eligible real cases in this collection to measure it properly here, which is listed as a concrete direction for future collection (Section 10).

The dominant failure mode shifted from “gave up on a benign miss” to “ran out of steps.” 6 of 24 runs exhausted the 35-step budget without concluding, a failure mode that was near-zero on the smaller, sparser synthetic graphs this architecture was originally tuned against. This is the clearest evidence in this data that step budget is not a free-standing tuning knob but scales with graph density in a way not yet characterised (Section 8).

7. Discussion

A minimal LLM agent, given only query tools and no view of the graph, found real, graph-confirmed attack paths on a genuine collected AD forest, without a single hallucinated hop across 24 runs, 4 models, and 3 vendors, at price points from \$0 to over a third of a dollar per run. That is the finding reported with the most confidence. Everything else in this dataset, which model is “best,” whether cost buys correctness, how the step budget should scale, is a *directional* signal from a small sample on one real topology, discussed below along with why it should be read that way rather than as a settled result.

8. Threats to Validity and Limitations

These are listed as specific, not generic, caveats; each ties to a concrete number above.

- **The verifier is not independently validated.** `verify_path` and the scorer share one implementation, checking a proposed path against the same collected graph the agent queried. This catches a model asserting an edge absent from the data, but it cannot catch an error the ingest pipeline and the verifier share (for example, a control edge the BloodHound parser mis-typed), nor does “graph-confirmed” mean “confirmed against the live domain”: no proposed path was re-tested against the actual GOAD-Light forest before it was destroyed. Every “correct” and “hallucination” figure in this paper should be read as *internally consistent with the collected data*, not as independently ground-truthed.
- **The rule-based baseline has no equivalent verification stage.** The agent must pass its own proposed path through a self-check before answering; the canonical shortest-path query it is compared against has no analogous second pass. This is not a fabricated advantage, a rule-based query has no natural “self-check” step to add, but it does mean the comparison is not between two methods at matched levels of engineering.
- **Correctness intervals overlap substantially across models.** gpt-4o-mini (50.0% [18.8 to 81.2]), claude-haiku-4-5 (16.7% [3.0 to 56.4]), and gpt-4o (16.7% [3.0 to 56.4]) all overlap at the 95% level; only gemini-flash-lite-latest's interval is clearly separated from the two lowest. The point estimates are reported because they are the actual observed rates, but a reader should not treat every ranking in Section

6.1 as statistically distinguishable: 6 runs per model is enough to see a pattern, not enough to certify one.

- **Step-budget scaling is an open question, not a solved lever.** 6 of 24 real-data runs hit the 35-step cap, versus near-zero on earlier, sparser synthetic graphs. Whether the budget a given graph needs scales linearly, sub-linearly, or worse with edge density is not known; the raised budget (15 to 35) is reported as a fix for a specific observed failure, not as evidence the relationship is understood.
- **Temperature 0.** All runs are deterministic-mode where the API supports it. This favours reproducibility over measuring run-to-run robustness; a variance sweep at non-zero temperature is future work, not something this dataset speaks to.
- **“Correctness” is graph-reachability correctness, not operational feasibility.** A path scored correct here is one the collected graph and its properties support; whether the access it assumes is actually still held at execution time, detection risk, and credential/ticket validity windows are not modelled. This is a narrower claim than “this path is exploitable in practice.”
- **Deliberately chosen, not randomly sampled, start users.** The 6 starts (Section 5) were picked to cover known path/no-path cases rather than sampled at random, which controls for a specific failure mode (hallucinating a path where none exists) at the cost of not being a representative draw over the graph's 33 real users.
- **One real topology, one collection pass per model.** All 24 runs are against a single GOAD-Light forest collected once; how much of the real-versus-synthetic gap in Section 6.1 is intrinsic to real AD structure, versus specific to this one topology, is not yet known.
- **Wide intervals throughout.** Every rate in Section 6 carries a 95% Wilson interval because the samples are small (6 runs per model); point estimates are reported with intervals precisely so a reader does not mistake, for example, 66.7% [30.0 to 90.3] for a precisely known rate.

9. Ethics

All experiments run against a lab that was built, owned, and destroyed for this purpose: five disposable cloud VMs running GOAD-Light, collected with SharpHound, BloodHound-CE, and Certipy, then torn down (after a disk snapshot) at the end of data collection. No live, third-party, or production network was touched at any point, and the collected data, credentials, hostnames, and topology, belongs entirely to a lab built for this purpose, not to any real organisation. The chat assistant and vulnerability checks built on the same verifier are read-only over the graph and refuse to assert anything the graph does not confirm. Any future live-collection work would run only against systems one owns or is explicitly authorised to test.

10. Conclusion and Future Work

An LLM attack-path agent was evaluated against a real, live-collected Active Directory forest, not synthetic data, across four models spanning three vendors and a roughly 100× price range. The agent found real, graph-confirmed paths with zero hallucinated hops across all 24 runs. Beyond that, the central finding is that the two most expensive models tested did not outperform a free one on this data (both tied for the lowest correctness observed), and that correctness dropped substantially relative to prior synthetic-graph results, which is attributed in part to the real forest's markedly denser, more ACL-heavy structure. Future work includes: a second real topology, to separate “real AD is different” from “this one topology is different”; enough inference-eligible real cases (Kerberoastable accounts, ADCS misconfigurations, unconstrained delegation) to properly re-measure advanced-case recall on real data, which this collection had too few of; a temperature greater than 0 variance sweep; and a principled study of how step budget should scale with graph density rather than the single manual adjustment reported here.

Data and Code Availability

The agent, shared verifier, evaluation harness, real-data collection manifest, and raw per-run logs (each archived with its git commit, model set, and timestamp) are released as open source at github.com/Crepc0/Ariadne under the MIT license. All tables and figures in this paper regenerate from the benchmark runner against the ingested real-data graph.